

# On Synthesizing a Consistent Operational Transformation Approach

Aurel Randolph, Hanifa Boucheneb, Abdessamad Imine, and Alejandro Quintero

**Abstract**—The operational transformation (OT) approach, used in many collaborative editors, allows a group of users to concurrently update replicas of a shared object and exchange their updates in any order. The basic idea is to transform any received update operation before its execution on a replica of the object. Concretely, OT consists of a centralized/decentralized integration procedure and a transformation function. In the context of decentralized integration, designing transformation functions for achieving convergence of object replicas is a critical and challenging issue. Indeed, the transformation functions proposed in the literature are all revealed inefficient. In this paper, we investigate the existence of transformation functions. From the theoretical point of view, two properties, named TP1 and TP2, are necessary and sufficient to ensure convergence. Using controller synthesis technique, we show that there are some transformation functions, which satisfy TP1 for the basic signatures of insert and delete operations. But, there is no transformation function, which satisfies both TP1 and TP2. Consequently, a transformation function which satisfies both TP1 and TP2 must necessarily have additional parameters in the signatures of some update operations. We propose, in this paper, a new transformation function and show formally that it ensures convergence.

**Index Terms**—Collaborative editors, operational transformation, proof of convergence, symbolic model checking, controller synthesis

## 1 INTRODUCTION

COLLABORATIVE editing systems (CESs for short) constitute a class of distributed systems where dispersed users interact by manipulating some shared objects like texts, images, graphics, XML documents, etc. To improve data availability, these systems are based on data replication. Each user has a local private copy of the shared object that he can access and update. The update operations executed locally are propagated to other users. The execution of these operations in different orders may lead to a divergence (object replicas are not identical). As an example, suppose two users  $u_1$  and  $u_2$  working on their own copies of a text containing the word “efecte”, starting at position 0. User  $u_1$  inserts ‘f’ at position 1, to change the word into “effecte”. Concurrently, user  $u_2$  deletes element at position 5 (i.e., the last ‘e’), to change the word into “effect”. Each user will receive an update operation that was applied on a different version of the text. Applying naively the received update operations will lead to divergent replicas (“effece” for user  $u_1$  and “effect” for user  $u_2$ , see Fig. 1). Moreover, users may generate concurrently conflicting or identical operations. The challenge in such situations is to ensure convergence of replicated data whilst preserving the intention of users.

Several approaches are proposed in the literature, to deal with the convergence of replicated data: Multi-Version

(MV) [1], Serialization-Resolution of Conflicts (SRC) [2], Commutative Replicated Data Type (CRDT) [3], [4], Operational Transformation (OT) [2], etc.

The multi-version approach, used in CVS, Subversion and ClearCase, is based on the paradigm “Copy-Modify-Merge” (see Fig. 3). In this approach, update operations made by a user are not automatically propagated to the others. They will be propagated only when the user explicitly calls the merge function. It would be interesting to propagate automatically, to all others, each update operation performed by a user. This is the basic idea of SRC.

To achieve convergence, SRC imposes to execute the operations in the same order at every site (see Fig. 4). Therefore, sites may have to undo and execute again operations, as they receive the final execution order of update operations. This order is determined by a central server fixed when the system is launched. For the previous example, this approach requires that sites of both users execute the two operations in the same order. However, even if we obtain an identical result in both sites, the execution order imposed by the central server may not correspond to the original intention of some user. For instance, executing, in both sites, the operation of  $u_1$  followed by the one of  $u_2$  results in the text “effece”, which is inconsistent with the intention of  $u_2$ .

The Commutative Replicated Data Type is a data type where all concurrent operations commute with each other [3], [4]. In such a case, to ensure convergence of replicas it suffices to respect the causality principle (i.e., whenever an operation  $o'$  is generated after executing another operation  $o$ ,  $o$  is executed before  $o'$  at every site). The main challenge of CRDT is designing commutative operations for the data type. The commonly used idea consists in associating a unique identifier with the position of each symbol, line or atom of the shared document and when an operation is generated, a unique identifier is also associated with the inserting/deleting position (see Fig. 5). The position identifiers do

- A. Randolph, H. Boucheneb and A. Quintero are with the Department of Computer and Software Engineering, École Polytechnique de Montréal, P.O. Box 6079, Station Centre-ville, Montréal, QC H3C 3A7, Canada. E-mail: {aurel.randolph, hanifa.boucheneb, alejandro.quintero}@polymtl.ca.
- A. Imine is with Lorraine University and INRIA Nancy-Grand-Est, Nancy, France. E-mail: abdessamad.imine@inria.fr.

Manuscript received 7 Mar. 2013; revised 27 Dec. 2013; accepted 10 Jan. 2014. Date of publication 24 Feb. 2014; date of current version 13 Mar. 2015.

Recommended for acceptance by S. Rajasekaran.

For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.

Digital Object Identifier no. 10.1109/TC.2014.2308203

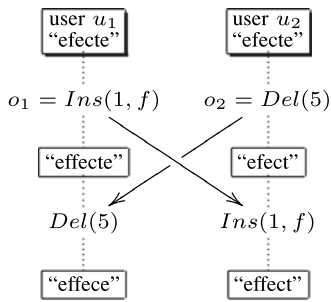


Fig. 1. Integration without transformation.

not change and are totally ordered with regard to  $<$ . Symbols, lines or atoms of the document appear in increasing order with regard to their identifiers. Let us apply this paradigm to the previous example. A unique identifier is associated with each symbol of the initial text:  $(e, 3) (f, 6) (e, 8) (c, 9) (t, 9.5) (e, 10)$ . A unique identifier between 3 and 6 is affected to position 1 of the operation of  $u_1$ . Let 4.5 be the selected identifier. The identifier affected to position 5 of the delete operation of  $u_2$  is 10. Both execution orders of operations of  $u_1$  and  $u_2$  lead to the text  $(e, 3) (f, 4.5) (f, 6) (e, 8) (c, 9) (t, 9.5)$ . CESs like TreeDoc [3], Logoot [5], Logoot-Undo [4] and WOOT [6] are based on CRDT paradigm. Managing position identifiers is a very important issue in this approach as the correctness is based on the uniqueness of position identifiers. Ensuring uniqueness may induce space and time overheads. Moreover, the identifier space increases regardless of the size of the document.

Unlike CRDT, in OT approach, proposed by Ellis and Gibbs [2], there is no need to associate a unique identifier with each symbol and the generated concurrent operations are not necessarily commutative. Their commutativity is forced by transformation of operations before their execution. More precisely, when a site receives an update operation, it is first transformed with regard to concurrent operations already executed on the site. The transformed operation is then executed on the local copy. This transformation aims at ensuring the convergence of copies even if users execute the same set of operations in different orders but with regard to causality principle. Concretely, OT consists of a centralized/decentralized integration procedure and a transformation function, called inclusive transformation (IT).

The integration procedure is in charge of executing update operations, broadcasting local update operations to other sites, receiving update operations from other sites, and determining transformations to be performed on a received operation before its execution. Several integration procedures have been proposed in the groupware research area, such as dOPT [2], adOPTed [7], SOCT2, 4 [8], [9], GOTO [10] and COT [11]. There are two kinds of integration procedures: centralized and decentralized. In the centralized integration procedures such as SOCT4 and COT, there is a central node which ensures that all concurrent operations are executed in the same order at all sites. In the decentralized integration procedures such as adOPTed, SOCT2 and GOTO, there is no central node and the operations may be executed in different orders by different sites. We focus, in

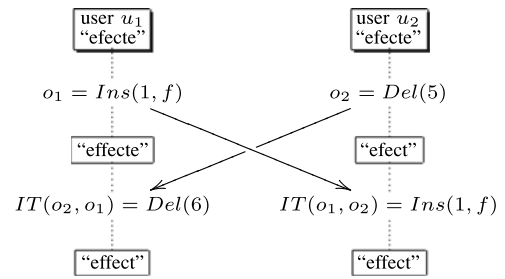


Fig. 2. Integration with transformation.

the following, on the decentralized integration procedures as they do not need any central server and then are more appropriate to P2P systems.

The IT function transforms an update operation with regard to another one. For the previous example, when  $u_1$  receives the operation of  $u_2$ , it is first transformed with regard to the local operation as follows:  $IT(Del(5), Ins(1, f)) = Del(6)$ . The deletion position is incremented because  $u_1$  has inserted a character at position 1, which is before the character deleted by  $u_2$ . Next, the transformed operation is executed on the local copy of  $u_1$ . In a similar way, when  $u_2$  receives the operation of  $u_1$ , it is transformed as follows before its execution on the local copy of  $u_2$ :  $IT(Ins(1, f), Del(5)) = Ins(1, f)$  (see Fig. 2). We can find, in the literature, several IT functions: Ellis's algorithm [2], Ressel's algorithm [7], Sun's algorithm [12], Suleiman's algorithm [13] and Imine's algorithm [14]. However, all these functions fail to ensure convergence [15], [16], [17], [18]. More precisely, from the theoretical point of view, to ensure data convergence, the IT function has to satisfy two properties named TP1 and TP2 [7]. Intuitively, IT satisfies TP1 means that IT forces commutativity of any pair of update operations executed in different order, on the same state. IT satisfies TP2 means that an update operation is transformed in the same manner with regard to two different but equivalent sequences of

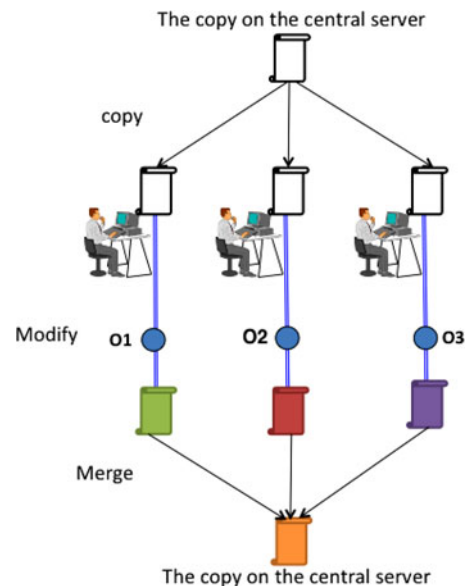


Fig. 3. MV approach.

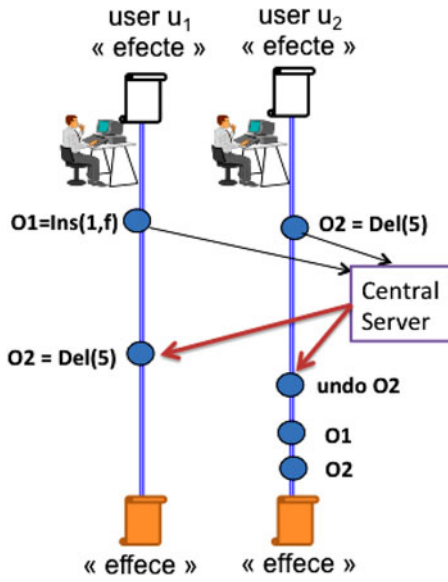


Fig. 4. SRC approach.

update operations. Some IT functions, proposed in the literature, satisfy TP1 but there is no IT function which satisfies TP2. To overcome this problem, some solutions impose a total ordering on the integration of operations, fixed in general by a central site. In this context, property TP1 is used to preserve the user intentions. For instance, in Fig. 4 both update operations are executed in the same order in both sites but the intention of  $u_2$  is not preserved. The transformation of the update operation of  $u_2$  with regards of the operation of  $u_1$ :  $IT(o_2, o_1) = Del(6)$  will preserve the intention of  $u_2$ . Such solutions are used in some tools such as Git,<sup>1</sup> Joint Emacs [7], CoWord [19], CoPowerPoint [19] and Google Wave.<sup>2</sup> However, an IT function which satisfies both TP1 and TP2 will be of great interest and value for developing interesting distributed applications in the context of P2P architectures.

In this paper, we first investigate the existence of IT functions, based on the classical signatures of update operations, which satisfy TP1 and TP2. Then, we propose a new IT function and show formally that it ensures convergence.

Section 2 is devoted to OT and IT functions proposed in the literature. For each IT function, we provide a counterexample for the convergence property. In Section 3, we show, using a controller synthesis technique, that for the classical signatures of update operations, there are some IT functions, which satisfy TP1 but there is no IT function, which satisfies both TP1 and TP2. Exploring the reasons of this failure identifies two problematic scenarios, which need two different transformation functions to ensure convergence. Consequently, a transformation function which satisfies TP1 and TP2 must have additional parameters in the operation signatures. Finding the appropriate parameters to be added to the signatures of the update operations, so as to ensure convergence with low overhead, is very challenging. Indeed, almost all IT functions, proposed in

the literature, are based on extending the signature of the insert operation by one or two parameters but fail to ensure data convergence. We propose, in Section 4, to extend the signature of the insertion with the number of symbols deleted before the inserting position and establish a new transformation algorithm. We end, this section, by proving formally, using a symbolic model checking technique, that our IT function satisfies both TP1 and TP2 and then ensures convergence. Conclusion goes in Section 5.

## 2 OPERATIONAL TRANSFORMATION APPROACH

### 2.1 Background

OT considers  $n$  sites, where each site has a copy of the shared object which is a finite sequence of elements from a data type  $\mathcal{A}$ . It is assumed here that the shared object can only be modified by the following primitive operations:

$$\mathcal{O} = \{Ins(p, c) | c \in \mathcal{A} \text{ and } p \in \mathbb{N}\} \cup \{Del(p) | p \in \mathbb{N}\} \cup \{Nop()\},$$

where  $Ins(p, c)$  inserts the element  $c$  at position  $p$ ;  $Del(p)$  deletes the element at position  $p$ , and  $Nop()$  is the idle operation that has null effect on the object.

Each site can concurrently update its copy of the shared object. Its local updates are then propagated to other sites. When a site receives an update operation, it is first transformed before its execution. Since the shared object is replicated, each site will own a local state  $l$  that is altered only by operations executed locally. The initial state of the shared object, denoted  $l_0$ , is the same for all sites. Let  $\mathcal{L}$  be the set of states. The function  $Do : \mathcal{O} \times \mathcal{L} \rightarrow \mathcal{L}$ , computes the state  $Do(o, l)$  resulting from applying operation  $o$  to state  $l$ . We denote by  $S = [o_1; o_2; \dots; o_m]$  an operation sequence and  $S = S_1 \bullet S_2$  a sequence obtained by concatenating sequences  $S_1$  and  $S_2$ . Applying an operation sequence to a state  $l$  is defined as follows: (i)  $Do^*([], l) = l$ , where  $[]$  is the empty sequence and (ii)  $Do^*(S \bullet [o], l) = Do(o, Do^*(S, l))$ ,  $S$  being a sequence of operations.

Two sequences of operations  $S_1$  and  $S_2$  are *equivalent*, denoted  $S_1 \equiv S_2$ , iff  $Do^*(S_1, l) = Do^*(S_2, l)$  for every state  $l$ .

OT consists of the integration procedure and the transformation function (IT function). The integration procedure is in charge of executing update operations, broadcasting local update operations to other sites, receiving update operations from other sites, and determining transformations to be performed on a received operation before its execution. The transformation function transforms an update operation  $o$  with regard to another update operation  $o'$  ( $IT(o, o')$ ). Let  $S = [o_1; o_2; \dots; o_m]$  be a sequence of operations. Transforming any update operation  $o$  with regard to  $S$ , denoted  $IT^*(o, S)$ , is recursively defined by:

$$IT^*(o, []) = o \text{ and}$$

$$IT^*(o, [o_1; o_2; \dots; o_m]) = IT^*(IT(o, o_1), [o_2; \dots; o_m]).$$

By definition:  $IT(Nop(), o) = Nop()$  and  $IT(o, Nop()) = o$  for every operation  $o$ .

### 2.2 Decentralized Integration Procedures

The integration procedure is based on two notions: concurrency and dependency of operations. Let  $o_1$  and  $o_2$  be

1. git.smc.com.

2. <http://www.waveprotocol.org/whitepapers/operational-transform>.

operations generated at sites  $i$  and  $j$ , respectively. Operation  $o_2$  is said *causally dependent* on  $o_1$ , denoted  $o_1 \rightarrow o_2$ , iff: (i)  $i = j$  and  $o_1$  was generated before  $o_2$ ; or, (ii)  $i \neq j$  and the execution of  $o_1$  at site  $j$  has happened before the generation of  $o_2$ . Operations  $o_1$  and  $o_2$  are said to be *concurrent*, denoted  $o_1 \parallel o_2$ , iff neither  $o_1 \rightarrow o_2$  nor  $o_2 \rightarrow o_1$ .

As a long established convention in OT-based collaborative editors [2], [10], the *timestamp vectors* are used to determine the causality and concurrency relations between operations. A timestamp vector is associated with each site and each generated operation. Every timestamp is a vector of integers with a number of entries equal to the number of sites. For a site  $j$ , each entry  $V_j[i]$  returns the number of operations generated at site  $i$  that have been already executed on site  $j$ . When an operation  $o$  is generated at site  $i$ , a copy  $W_o$  of  $V_i$  is associated with  $o$  before its broadcast to other sites. The entry  $V_i[i]$  is then incremented by 1. Once  $o$  is received at site  $j$ , if the local vector  $V_j$  “dominates”<sup>3</sup>  $W_o$ , then  $o$  is ready to be executed on site  $j$ . In this case,  $V_j[i]$  will be incremented by 1 after the execution of  $o$ . Otherwise, the  $o$ ’s execution is delayed until the causality condition holds. Let  $W_{o_1}$  and  $W_{o_2}$  be timestamp vectors of  $o_1$  and  $o_2$ , respectively. Using these timestamp vectors, the causality and concurrency relations are defined as follows:

- (i)  $o_1 \rightarrow o_2$  iff  $W_{o_2}$  dominates  $W_{o_1}$  and  $W_{o_1} \neq W_{o_2}$ .
- (ii)  $o_1 \parallel o_2$  iff neither  $W_{o_1}$  dominates  $W_{o_2}$  nor  $W_{o_2}$  dominates  $W_{o_1}$ .

In the decentralized integration procedures, every site generates operations sequentially and stores these operations in a stack also called a *history*. When a site receives a remote operation  $o$ , the integration procedure executes the following steps:

- 1) From the local history  $S$ , it determines the equivalent sequence  $S'$  that is the concatenation of two sequences  $S_h$  and  $S_c$  where (i)  $S_h$  contains all operations happened before  $o$  (according to the causality relation), and (ii)  $S_c$  consists of operations that are concurrent to  $o$ .
- 2) It calls the transformation component in order to get operation  $o'$  that is the transformation of  $o$  according to  $S_c$  (i.e.,  $o' = IT^*(o, S_c)$ ).
- 3) It executes  $o'$  on the current state and then adds  $o'$  to local history  $S$ .

The integration procedure allows history of executed operations to be built on every site, provided that the causality relation is preserved. When all sites have executed the same set of operations (stable states), their histories are not necessarily identical because the concurrent operations may be executed in different orders. Nevertheless, they must be equivalent in the sense that they must lead to the same final state.

### 2.3 Inclusive Transformation Functions

We can find, in the literature, several IT functions: *Ellis’s* algorithm [2], *Ressel’s* algorithm [7], *Sun’s* algorithm [12], *Suleiman’s* algorithm [13] and *Imine’s* algorithm [14]. They

3. Let  $V_1$  and  $V_2$  be two timestamp vectors. We say that  $V_1$  dominates  $V_2$  iff for each site  $i$ ,  $V_1[i] \geq V_2[i]$ .

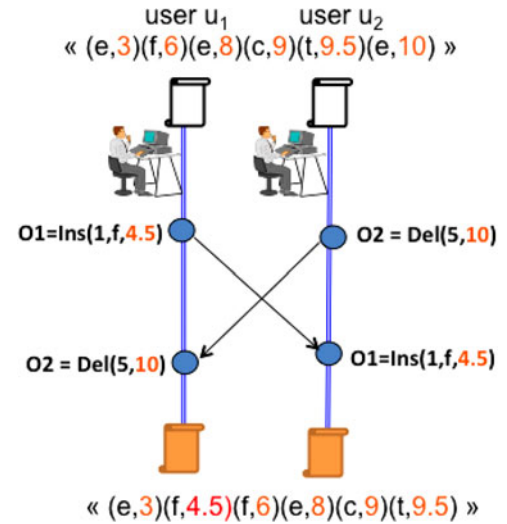


Fig. 5. CRDT approach.

differ in the manner that conflict situations are managed. A conflict situation occurs when two concurrent operations insert, at the same position, different characters. To deal with such conflicts, all these algorithms, except the one proposed by Sun et al., add some extra parameters to the insert operation signature.

- 1) *Ellis’s algorithm*. Ellis and Gibbs [2] are the pioneers of OT approach. They extend operation *Ins* with another parameter *pr* representing its priority. Concurrent operations always have different priorities. The four transformation cases for *Ins* and *Del* proposed by Ellis and Gibbs are illustrated in the appendix, which can be found on the Computer Society Digital Library at <http://doi.ieeecomputersociety.org/10.1109/TC.2014.2308203>.
- 2) *Ressel’s algorithm*. Ressel et al. [7] proposed an algorithm that provides two modifications to Ellis’s algorithm. The first modification consists in replacing priority parameter *pr* by another parameter *u*, which is simply the *identifier* of the issuer site. Similarly, *u* is used for tie-breaking when a conflict occurs between two concurrent insert operations. As for the second modification, it concerns how a pair of insert operations is transformed. When two concurrent insert operations add at the same position two (identical or different) elements, only the insertion position of operation having a higher identifier is incremented. In other words, both elements are inserted even if they are identical. This is the opposite to the solution proposed by Ellis and Gibbs, which keeps only one element in case of identical concurrent insertions. Apart from these modifications, the other cases remain similar to those of Ellis and Gibb. The transformation cases given by the algorithm of Ressel et al. [7] can be found in the appendix, available in the online supplemental material.
- 3) *Sun’s algorithm*. Sun et al. [12] have designed another IT algorithm, which is slightly different in the sense that it is defined for stringwise operations. Indeed,

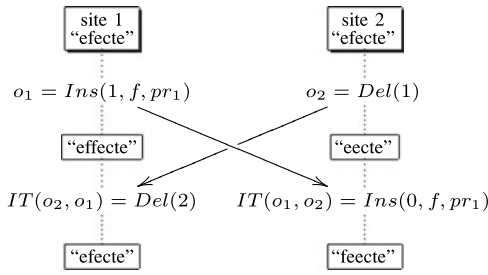


Fig. 6. Violation of TP1 for Ellis's IT.

the following operations are used:  $Ins(p, s, l)$  (insert string  $s$  of length  $l$  at position  $p$ ) and  $Del(p, l)$  (delete string of length  $l$  from position  $p$ ). To compare with other IT algorithms, we suppose that  $l = 1$  for all update operations. The IT function in this case is reported in the appendix, available in the online supplemental material.

- 4) *Suleiman's algorithm.* Suleiman et al. [13] proposed another solution that modifies the signature of insert operation by adding two parameters  $av$  and  $ap$ . For an insert operation  $Ins(p, c, av, ap)$ ,  $av$  contains operations that have deleted a character before the insertion position  $p$ . The set  $ap$  contains operations that have removed a character after or at position  $p$ . When an insert operation is generated, the parameters  $av$  and  $ap$  are empty. They will be filled during transformation steps. The IT function of Suleiman et al. is given in the appendix, available in the online supplemental material. To resolve the conflict between two concurrent insert operations  $Ins(p, c_1, av_1, ap_1)$  and  $Ins(p, c_2, av_2, ap_2)$ , three cases are possible:

- 1)  $(av_1 \cap av_2) \neq \emptyset$ : character  $c_2$  is inserted before character  $c_1$ ,
- 2)  $(ap_1 \cap av_2) \neq \emptyset$ : character  $c_2$  is inserted after character  $c_1$ ,
- 3)  $(av_1 \cap ap_2) = (ap_1 \cap av_2) = \emptyset$ : in this case, characters  $c_1$  and  $c_2$  are compared (for instance according to the lexicographic order) to choose the one to be added before the other. Like the site identifiers and priorities, parameters  $av$ ,  $ap$ , comparison of characters are used to tie-break conflict situations. Note that when two concurrent operations insert the same character (e.g.,  $c_1 = c_2$ ) at the same position, only one is executed. The other one is ignored by returning the idle operation  $Nop()$ . In other words, like the solution of Ellis and Gibb [2], only one character is kept.

- 5) *Imine's algorithm.* In [14], Imine et al. proposed another IT algorithm which again enriches the signature of insert operation with parameter  $ip$  which is the initial (or the original) insertion position given at the generation stage. Thus, when transforming a pair of insert operations having the same current position, they compare first their initial positions in order to recover the position relation at the generation phase. If the initial positions are identical, then like Suleiman et al. [13] they compare symbols to tie-break an eventual conflict. The

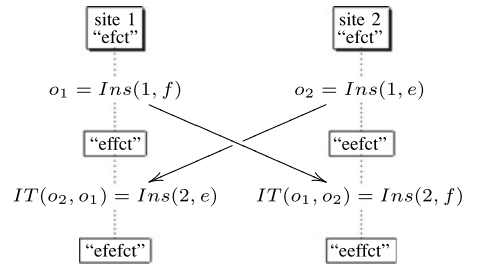


Fig. 7. Violation of TP1 for Sun's IT.

IT function of Imine can be found in the appendix, available in the online supplemental material.

## 2.4 Consistency Criteria

An OT-based collaborative editor is *consistent* iff it satisfies the following properties:

- 1) *Causality preservation.* if  $o_1 \rightarrow o_2$  then  $o_1$  is executed before  $o_2$  at all sites.
- 2) *Convergence.* when all sites have performed the same set of updates, the copies of the shared document are identical.

To preserve the causal dependency between update operations, timestamp vectors are used. In [7], the authors have established two properties *TP1* and *TP2* that are necessary and sufficient to ensure data convergence for *any number* of operations executed in *arbitrary order* on copies of the same object: For all  $o_1$ ,  $o_2$  and  $o_3$  pairwise concurrent operations defined on the same state (initial state or state reached from the initial state by executing equivalent sequences):

- *TP1:*  $[o_1; IT(o_2, o_1)] \equiv [o_2; IT(o_1, o_2)]$ .
- *TP2:*  $IT^*(o_3, [o_1; IT(o_2, o_1)]) = IT^*(o_3, [o_2; IT(o_1, o_2)])$ .

Property *TP1* defines a *state identity* and ensures that if  $o_1$  and  $o_2$  are concurrent, the effect of executing  $o_1$  before  $o_2$  is the same as executing  $o_2$  before  $o_1$ . Property *TP2* ensures that transforming  $o_3$  along equivalent and different operation sequences will give the same operation. By abuse of language, an IT function satisfying properties *TP1* and *TP2* is said to be consistent.

Accordingly, by these properties, it is not necessary to enforce a global total order between concurrent operations because data divergence can always be repaired by operational transformation. However, finding an IT function that satisfies *TP1* and *TP2* is considered to be a hard task, because this proof is often unmanageably complicated.

All IT functions proposed, in the literature, do not ensure data converge [15], [17], [18]. We report, in the following, a counterexample for each IT function.

IT functions of Ellis and Sun do not satisfy the property *TP1* (see Figs. 6 and 7) [14]. The pairs of concurrent operations violating *TP1* are  $(o_1 = Ins(1, f, pr_1), o_2 = Del(1))$  and  $(o_1 = Ins(1, f), o_2 = Ins(1, e))$ , respectively.

Suleiman's IT satisfies neither *TP1* nor *TP2* [14], [18]. The counterexample for *TP1* is given by the pair of operations  $(o'_1 = Ins(2, f, \{o_3\}, \{o_5\}), o'_2 = Ins(2, c, \{o_5\}, \{o_3\}))$ . The corresponding scenario, reported at Fig. 8, consists of four users  $u_1, u_2, u_3$  and  $u_4$  on different sites. Users  $u_1, u_2$  and  $u_3$  have generated and executed locally sequences  $S_1 = [o_1 =$

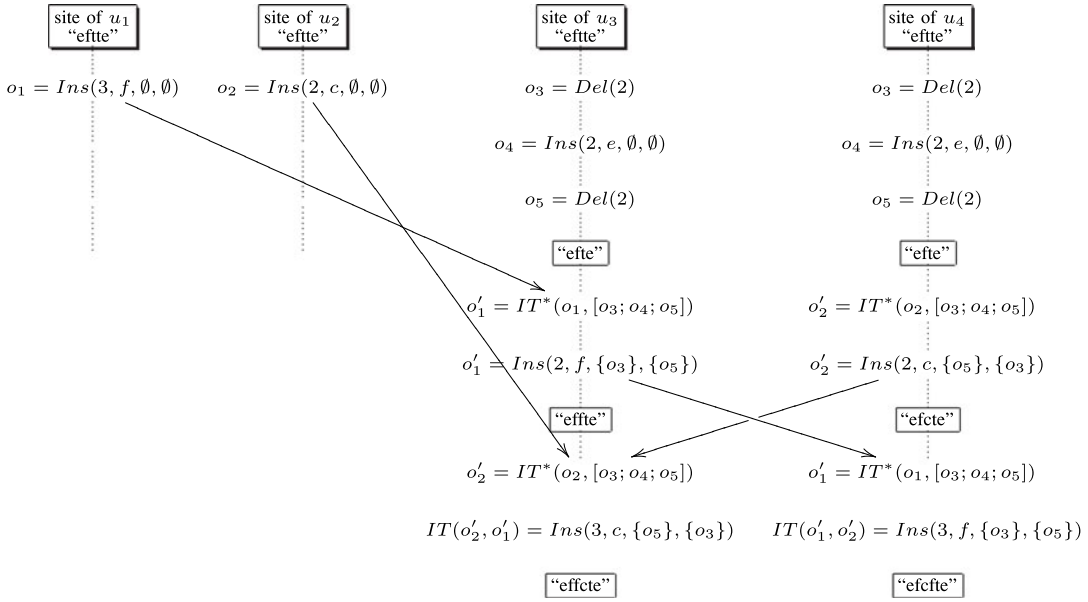


Fig. 8. Violation of TP1 for Suleiman's IT.

$Ins(3, f, \emptyset, \emptyset)$ ,  $S_2 = [o_2 = Ins(2, c, \emptyset, \emptyset)]$  and  $S_3 = [o_3 = Del(2); o_4 = Ins(2, e, \emptyset, \emptyset); o_5 = Del(2)]$ , respectively. Then, user  $u_3$  receives successively operations  $o_1$  and  $o_2$ . User  $u_4$  receives consecutively operations of  $S_3$ ,  $o_2$  and  $o_1$ . The IT function of Suleiman fails to ensure convergence. Property TP1 is violated for  $o'_1 = IT^*(o_1, S_3) = Ins(3, f, \{o_3\}, \{o_5\})$  and  $o'_2 = IT^*(o_2, S_3) = Ins(2, c, \{o_5\}, \{o_3\})$ .

Ressel's IT does not satisfy TP2 but satisfies TP1 [14], [18]. In Fig. 9, we report a scenario violating property TP2 for the triplet of concurrent operations ( $o_1 = Del(1), o_2 = Ins(2, c_2, u_2), o_3 = Ins(1, c_3, u_3)$ ).

Imine's IT function satisfies TP1 but does not satisfy TP2. In Fig. 10, we report a scenario violating TP2. In this scenario, there are four users  $u_1, u_2, u_3$  and  $u_4$  on different sites. Users  $u_1, u_2$  and  $u_3$  have generated sequences  $S_1 = [o_1 = Del(2)]$ ,  $S_2 = [o_0 = Del(2); o_2 = Ins(2, c, 2)]$  and  $S_3 = [o_3 = Ins(2, e, 2)]$ , respectively. User  $u_2$  executes operations  $o_0$  and  $o_2$  then it receives successively operations  $o_1$  and  $o_3$ . User  $u_4$  receives successively operations  $o_0, o_1, o_2$  and  $o_3$ . For this scenario, the IT function of Imine fails to ensure convergence for copies of users  $u_2$  and  $u_4$ . The property TP2 is violated for  $o'_1 = IT(o_1, o_0), o_2$  and  $o'_3 = IT(o_3, o_0)$  (see Fig. 10).

## 2.5 Avoiding Property TP2

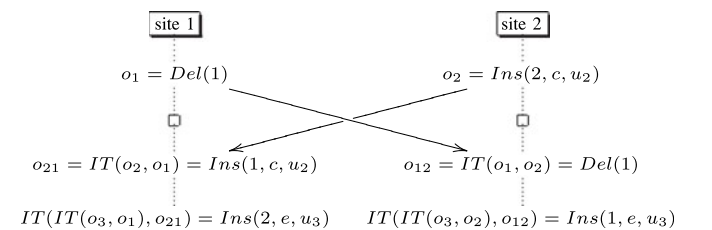
The violation of property TP2 gave birth to several works that have tried to avoid this problem, often at the expense of genericity and efficiency. These works may be categorized in two approaches.

The first approach falls in the category of works, such as GOT [12], SOCT4 [9] and COT [11], which enforce a total order on operations in order to maintain the same transformation path at all sites. Although these algorithms ensure the data convergence, they do not allow a high concurrency degree because of the global order of execution. In the second approach, some works build a particular class of transformation paths. For instance, OPTIC [16], [20] and ABT [21] integrate by transformation remote operations on logs organized in such a way that insertion operations are

always before deletion operations. Building these transformations needs, unfortunately, to devise new integration algorithms. Other works are based on changing the semantics of the delete operation [22]. The delete operation does not remove symbols but make them invisible (see Section 4.5 for comparison with the approach proposed here).

## 3 CONTROLLER SYNTHESIS OF CONSISTENT IT FUNCTION

In [15], [17], [18], the authors used a theorem proving or a model checking technique to verify whether or not a given IT is consistent. They showed that all IT functions, proposed in the literature, do not ensure consistency and a counterexample is provided for each IT function. From this fact, a question arises concerning the existence of consistent IT functions. Model checking and theorem proving are useful to prove whether or not a given system satisfies some properties but not appropriate to verify whether or not there exist a system, which satisfies some properties. The controller synthesis techniques solve a more general problem than the model checking and theorem proving techniques, since they verify whether or not the system can be modified (forced) so as to meet the properties of interest. In such a framework, the system consists, in general, of controllable and uncontrollable transitions. The control objective is to find, if it exists, a strategy to force the properties of interest, by choosing appropriately


 Fig. 9. Violation of TP2 for Ressel's IT (in case  $u_2 < u_3$ ).

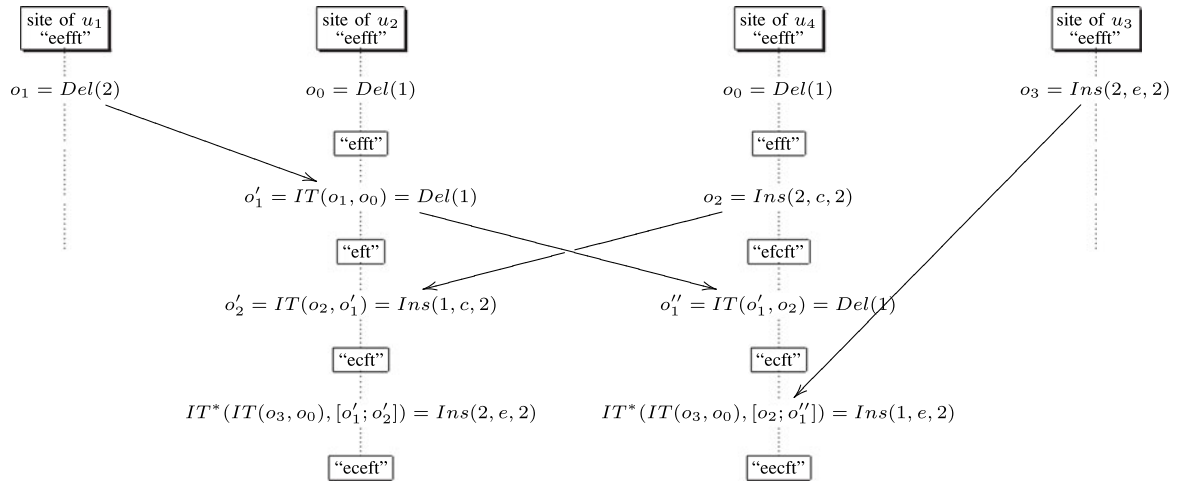


Fig. 10. Violation of TP2 for Imine's IT.

controllable actions to be executed, no matter what uncontrollable transitions are executed.

We are interested in applying the principle of controller synthesis to design an IT function which satisfies properties TP1 and TP2. The idea is to look for consistent IT in the set of allowed functions. There are four allowed transformations for each operation  $o$ : 1) increment its position, 2) decrement its position, 3) transform it into an idle operation ( $Nop()$ ), and 4) leave it unchanged. We first investigate whether or not there exist in such a set, some IT functions which satisfy property TP1. If it is the case, we investigate whether or not there exist some IT functions, among those satisfying TP1, which also satisfy TP2.

For these investigations, we use the game automata formalism 'à la UPPAAL' [23]. A game automaton is an automaton with two kinds of transitions: controllable and uncontrollable. Each transition has a source location, a destination location and is annotated with selections, guards and blocks of actions. Selections bind non-deterministically a given identifier to every value in a given range. The other labels of a transition are within the scope of this binding. A state is defined by the current location and the current values of all variables. A transition is enabled in a state iff the current location is the source location of the transition and its guard evaluates to true. The firing of the transition consists in reaching its destination location and executing atomically its block of actions. The side effect of this block changes the state of the system. To force some properties, the enabled transitions that are controllable can be delayed or simply ignored. However, the uncontrollable transitions can neither be delayed nor ignored.

### 3.1 Do There Exist IT Functions Which Satisfy TP1?

An IT function satisfies property TP1 iff for any pair of concurrent operations  $o_1$  and  $o_2$  defined on the same state, it holds that  $[o_1; IT(o_2, o_1)] \equiv [o_2; IT(o_1, o_2)]$ . To verify whether or not there are some IT functions which satisfy property TP1, we have represented in the game automaton, depicted at Fig. 11, the generation of operations  $o_1$  and  $o_2$ , the computation of  $IT(o_1, o_2)$  and  $IT(o_2, o_1)$ , and the verification of  $[o_1; IT(o_2, o_1)] \equiv [o_2; IT(o_1, o_2)]$ . The generation of operations is specified by the uncontrollable transition  $(s_0, s_1)$ , since we

have no control on the kinds of operations generated by users. The operational transformations and the verification of TP1 are represented by the controllable transition  $(s_1, s_2)$ . The model starts by selecting two operations  $o_1$  and  $o_2$ . The domain of operations is fixed so as to cover all cases of transformations. Afterwards, the model chooses two transformations to apply to  $o_1$  with regard to  $o_2$  and  $o_2$  with regard to  $o_1$  and applies them by invoking function  $IT1$ . Function  $IT1(o_1, o_2, o_{12}, IsNop, ip_1)$  returns in  $o_{12}$  the result of transformation of  $o_1$  with regard to  $o_2$ . If  $IsNop = true$  then  $o_{12} = Nop()$ , otherwise the transformation of  $o_1$  consists in updating the parameter position ( $o_{12}.p = o_1.p + ip_1$ ). It means that four possibilities are offered for transforming an operation  $o_1$  with regard to another operation  $o_2$ :  $Nop()$ , decrementing, maintaining, or incrementing the position of  $o_1$ . Finally, the model verifies whether or not the property TP1 is satisfied. No matter what operations  $o_1$  and  $o_2$  generated by the uncontrollable transition, the controller synthesis

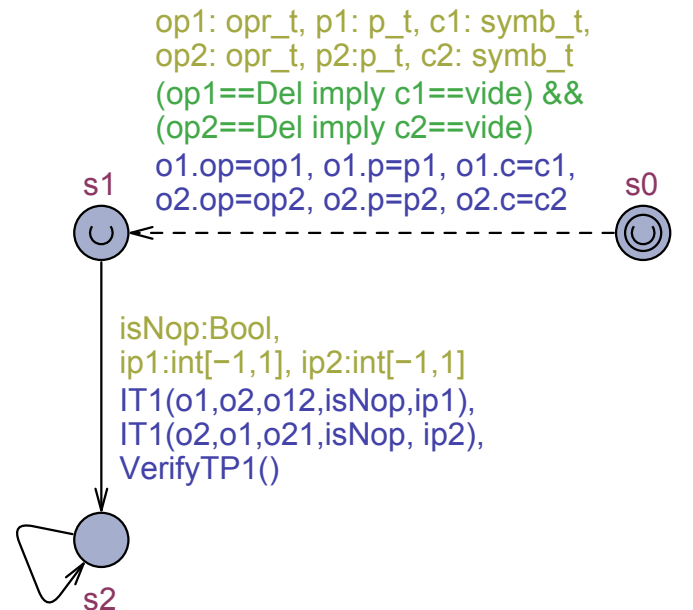


Fig. 11. Synthesize an IT for TP1.

TABLE 1  
IT Functions Supplied by *Uppaal-Tiga* for TP1 and Classical Signatures of Update Operations

| $o_1$           | $o_2$           | $Cnd(p_1, p_2, c_1, c_2)$    | $IT(o_1, o_2)$      | $IT(o_2, o_1)$      |
|-----------------|-----------------|------------------------------|---------------------|---------------------|
| $Ins(p_1, c_1)$ | $Ins(p_2, c_2)$ | $p_1 < p_2$                  | $Ins(p_1, c_1)$     | $Ins(p_2 + 1, c_2)$ |
| $Ins(p_1, c_1)$ | $Ins(p_2, c_2)$ | $p_1 = p_2 \wedge c_1 < c_2$ | $Ins(p_1 + 1, c_1)$ | $Ins(p_2, c_2)$     |
| $Ins(p_1, c_1)$ | $Ins(p_2, c_2)$ | $p_1 = p_2 \wedge c_1 < c_2$ | $Ins(p_1, c_1)$     | $Ins(p_2 + 1, c_2)$ |
| $Ins(p_1, c_1)$ | $Ins(p_2, c_2)$ | $p_1 = p_2 \wedge c_1 = c_2$ | $Ins(p_1 + 1, c_1)$ | $Ins(p_2 + 1, c_2)$ |
| $Ins(p_1, c_1)$ | $Ins(p_2, c_2)$ | $p_1 = p_2 \wedge c_1 = c_2$ | $Ins(p_1, c_1)$     | $Ins(p_2, c_2)$     |
| $Ins(p_1, c_1)$ | $Ins(p_2, c_2)$ | $p_1 = p_2 \wedge c_1 = c_2$ | $Nop()$             | $Nop()$             |
| $Del(p_1)$      | $Del(p_2)$      | $p_1 < p_2$                  | $Del(p_1)$          | $Del(p_2 - 1)$      |
| $Del(p_1)$      | $Del(p_2)$      | $p_1 = p_2$                  | $Del(p_1 - 1)$      | $Del(p_2 - 1)$      |
| $Del(p_1)$      | $Del(p_2)$      | $p_1 = p_2$                  | $Del(p_1 + 1)$      | $Del(p_2 + 1)$      |
| $Del(p_1)$      | $Del(p_2)$      | $p_1 = p_2$                  | $Del(p_1)$          | $Del(p_2)$          |
| $Del(p_1)$      | $Del(p_2)$      | $p_1 = p_2$                  | $Nop()$             | $Nop()$             |
| $Ins(p_1, c_1)$ | $Del(p_2)$      | $p_1 < p_2$                  | $Ins(p_1, c_1)$     | $Del(p_2 + 1)$      |
| $Ins(p_1, c_1)$ | $Del(p_2)$      | $p_1 = p_2$                  | $Ins(p_1, c_1)$     | $Del(p_2 + 1)$      |
| $Del(p_1)$      | $Ins(p_2, c_2)$ | $p_1 < p_2$                  | $Del(p_1)$          | $Ins(p_2 - 1, c_2)$ |
| $Del(p_1)$      | $Ins(p_2, c_2)$ | $p_1 = p_2$                  | $Del(p_2 + 1)$      | $Ins(p_1, c_1)$     |

aims to force property TP1 by choosing appropriately the operational transformations.

We have used the tool *Uppaal-Tiga* [23] to verify whether or not there exist some IT functions, which satisfy TP1. The safety control objective for TP1 is  $AG\ TP1$ , where  $TP1$  is defined in the model as a boolean variable whose value is *true* while the property TP1 is satisfied. The boolean variable  $TP1$  is set to false by the function *VerifyTP1* if  $[o_1; IT(o_2, o_1)] \neq [o_2; IT(o_1, o_2)]$ . *Uppaal-Tiga* concludes that the property is satisfied, which means that there is, at least, a strategy to force property TP1. We report in Table 1 the different IT functions (satisfying TP1) extracted from the output file of the tool *verifytga* of *Uppaal-Tiga*.

Note that even if some operational transformations satisfy TP1, they are unacceptable from the semantic point of view. For instance, if  $p_1 = p_2$ , the operational transformations  $IT(Del(p_1), Del(p_2)) = Del(p_1 - 1)$ ,  $IT(Del(p_1), Del(p_2)) = Del(p_1)$  and  $IT(Del(p_1), Del(p_2)) = Del(p_1 + 1)$  mean that if two users generate concurrently the same delete operation, two symbols will be deleted in each site, which is unacceptable from the semantic point of view. The only operational transformation which makes sense for this case is  $IT(Del(p_1), Del(p_2)) = Nop()$ . It means that only the symbol at position  $p_1$  is deleted in each site. After eliminating these incoherent operational transformations, there remain two possibilities for  $IT(Ins(p_1, c_1), Ins(p_2, c_2))$ ,  $p_1 = p_2, c_1 \neq c_2$ , and three possibilities for  $IT(Ins(p_1, c_1), Ins(p_2, c_2))$ ,  $p_1 = p_2, c_1 = c_2$ . Therefore, we can extract six IT functions which satisfy TP1. These IT functions differ in the way that conflicting operations are managed.

### 3.2 Do There Exist IT Functions Which Satisfy TP1 and TP2?

An IT function satisfies property  $TP2$  iff for any triplet of pairwise concurrent operations  $o_1, o_2$  and  $o_3$  defined on the same state, it holds that  $IT(IT(o_3, o_1), IT(o_2, o_1)) = IT(IT(o_3, o_2), IT(o_1, o_2))$ . To verify whether or not there are some IT functions which satisfy properties TP1 and TP2, we have used the game automaton depicted at Fig. 12. This model starts by selecting an IT function, which satisfies property TP1 (the range of *chooseIT* corresponds to the six IT functions satisfying TP1). Afterwards, it selects

three operations  $o_1, o_2$  and  $o_3$ , and performs the transformations needed to verify TP2. Function  $IT2(o_1, o_2, o_{12})$  applies the selected IT function to  $o_1$  with regard to  $o_2$  and returns the result of this transformation in  $o_{12}$ . Finally, the model calls function *VerifyTP2*. The control aims to force to choose the appropriate IT function so as to satisfy property TP2. The control objective is specified by the CTL formula  $AG\ TP2$ , where  $TP2$  is a boolean variable whose value is *true* while the property TP2 is satisfied. This variable is set to false by the function *VerifyTP2* if  $IT(IT(o_3, o_1), IT(o_2, o_1)) \neq (IT(o_3, o_2), IT(o_1, o_2))$ .

*Uppaal-Tiga* concludes that the property  $AG\ TP2$  cannot be forced, which means that there is no strategy to force property TP2. In other words, there is no IT function, based on classical parameters of delete and insert operations, which satisfies both TP1 and TP2. We investigated

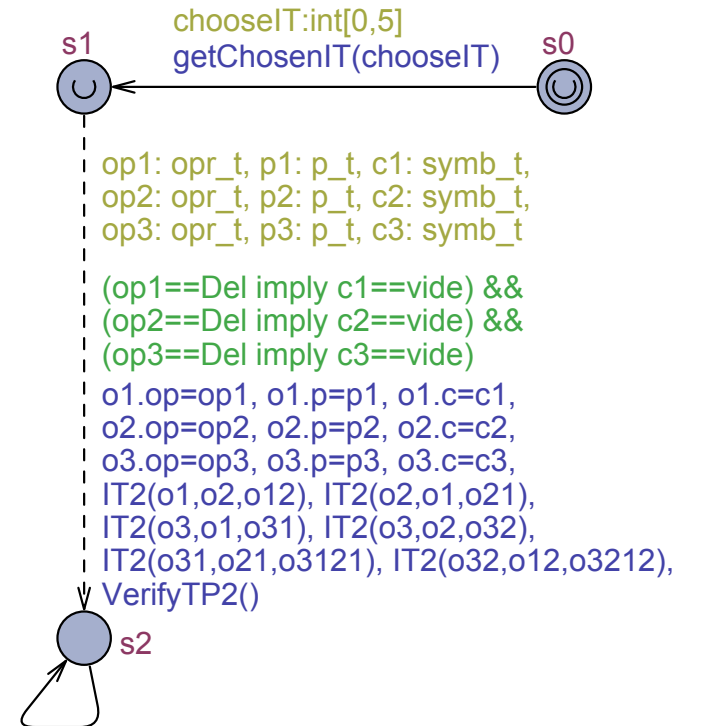


Fig. 12. Synthesize a consistent IT.

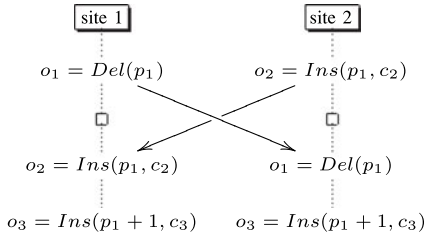


Fig. 13. Scenario 1.

why a consistent IT function does not exist for the basic parameters of delete and insert operations. This investigation led to the identification of two symbolic pairwise scenarios which prevent a consistent IT function from being obtained. We report in Figs. 13 and 14 these two pairwise sequences named scenarios 1 and 2, respectively. For scenario 1, to verify TP2, the performed transformations are:

$$\begin{aligned} o_{21} &= IT(o_2, o_1) = IT(Ins(p_1, c_2), o_1) = Ins(p_1, c_2), \\ o_{12} &= IT(o_1, o_2) = IT(Del(p_1), Ins(p_1, c_2)) = Del(p_1 + 1), \\ o_{31} &= IT(o_3, o_1) = Ins(p_1, c_3), \\ o_{32} &= IT(o_3, o_2) = Ins(p_1 + 2, c_3), \end{aligned}$$

$$IT(o_{32}, o_{12}) = IT(Ins(p_1 + 2, c_3), Del(p_1 + 1)) = Ins(p_1 + 1, c_3)$$

and  $IT(o_{31}, o_{21}) = IT(Ins(p_1, c_3), Ins(p_1, c_2))$ . For the last transformation, we have different possibilities (see Table 1). To satisfy TP2, we must choose  $IT(Ins(p_1, c_3), Ins(p_1, c_2)) = Ins(p_1 + 1, c_3)$ .

For scenario 2, the performed transformations are:

$$\begin{aligned} o_{21} &= IT(o_2, o_1) = Ins(p_1, c_2), \quad o_{12} = IT(o_1, o_2) = Del(p_1), \\ o_{31} &= IT(o_3, o_1) = Ins(p_1, c_3), \\ o_{32} &= IT(o_3, o_2) = Ins(p_1, c_3), \\ IT(o_{32}, o_{12}) &= IT(Ins(p_1, c_3), Del(p_1)) = Ins(p_1, c_3) \text{ and } IT(o_{31}, o_{21}) = IT(Ins(p_1, c_3), Ins(p_1, c_2)). \end{aligned}$$

To satisfy TP2, for the last operational transformation, we must use  $IT(Ins(p_1, c_3), Ins(p_1, c_2)) = Ins(p_1, c_3)$ .

Consequently, a consistent IT function, if it exists, must have additional parameters in its operation signatures. We have seen, in the previous section, different IT functions based on extending the insert signature with priority, issuer site, initial position or sets of delete operations before and after the inserting position. We have provided for each IT function a divergent scenario violating either TP1 or TP2. It means that the suggested additional parameters are insufficient or inappropriate to ensure convergence. Indeed, adding priority (as in Ellis's IT) or owner identifier (as in Ressel's IT) to the insert signature

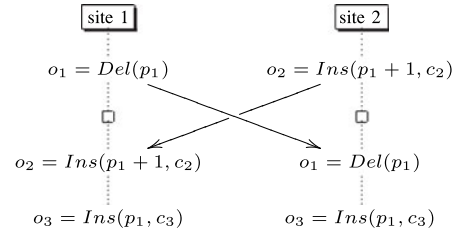


Fig. 14. Scenario 2.

fails to ensure convergence for scenarios 1 and 2. Scenario 1 violates TP1 for Ellis's IT (see Fig. 6). Scenario 2 violates TP2 for Ressel's IT (see Fig. 9). For Suleiman's IT and Imine's IT, scenarios 1 and 2 satisfy TP1 and TP2 but the added parameters introduce other cases of divergence (see Figs. 8 and 10). All these failed tentatives show that designing a consistent IT function is a hard task.

The main difference between both scenarios resides in the position of the deleted symbol relatively to the symbols inserted by  $o_2$  and  $o_3$  (see Figs. 13 and 14). In scenario 1, the deleted symbol is before the inserting position of  $o_3$ , whereas, in scenario 2, it is before the inserting position of  $o_2$ . Extending the signature of the insert operation with the number of symbols deleted before its position allows the definition of appropriate transformations so as both scenarios 1 and 2 satisfy TP1 and TP2 (see Figs. 15 and 16). Indeed, in scenario 1, the symbol of  $o_{31} = IT(o_3, o_1) = Ins(p_1, c_3, 1)$  should precede the one of  $o_{21} = Ins(p_1, c_2, 0)$ , as  $o_{31}$  has the biggest number of symbols deleted before its position ( $1 > 0$ ). In scenario 2,  $o_{21} = IT(o_2, o_1) = Ins(p_1, c_2, 1)$  has the greatest number of symbols deleted before its position. Its symbol should be inserted after the one of  $o_{31} = Ins(p_1, c_3, 0)$ . The idea of our IT function is to use this extra parameter to deal with conflicting operations. In the following, we propose based on this idea an IT function and show formally its consistency.

## 4 A CONSISTENT IT FUNCTION

### 4.1 Extending the Insert Signature with an Extra Parameter

We propose to add a new parameter, named  $nd$ , to the insert operation signature. This extra parameter is filled with the number of symbols deleted before the inserting position. When an insert operation  $o$  is generated, its parameter  $nd$  is set appropriately to the number of symbols deleted before the position of  $o$ . Afterwards, this parameter is incremented

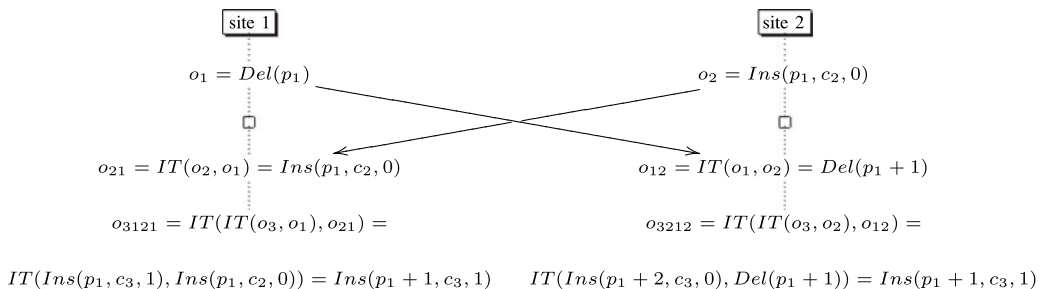


Fig. 15. Applying our IT to scenario 1.

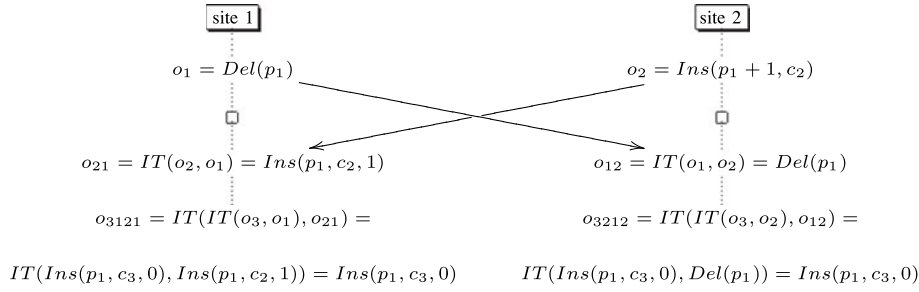


Fig. 16. Applying our IT to scenario 2.

whenever it is transformed with regard to a delete operation whose position is before its inserting position.

**Definition 1.** Let  $S$  be a sequence of operations executed on the shared document before generating an insert operation  $o$ . Let  $p$  be the inserting position of  $o$ , we denote  $ND(S, p)$  the number of symbols deleted before  $p$  by the sequence  $S$ . The parameter  $nd$  of  $o$  set to  $ND(S, p)$  is inductively defined as follows:

$$ND(S, p) = \begin{cases} 0, & \text{if } S = [], \\ ND(S', p), & \text{if } S = S' \bullet [o'] \wedge p < p', \\ nd', & \text{if } S = S' \bullet [Ins(p', c', nd')] \wedge p = p', \\ ND(S', p - 1), & \text{if } S = S' \bullet [Ins(p', c', nd')] \wedge p > p', \\ ND(S', p + 1) + 1, & \text{if } S = S' \bullet [Del(p')] \wedge p \geq p', \end{cases}$$

where  $o' \in \{Ins(p', c', nd'), Del(p')\}$ .

Intuitively, if  $S$  is empty, it is trivial that there is no symbol deleted by  $S$ . Then,  $ND([], p) = 0$ . For  $S = S' \bullet [Ins(p', c', nd')]$ , the symbol of  $Ins(p', c', nd')$  is inserted at position  $p'$ . If  $p < p'$ , the execution of  $Ins(p', c', nd')$  does not affect the part before position  $p'$  (i.e.,  $ND(S, p) = ND(S', p)$ ). If  $p = p'$ , the symbol  $c$  is inserted at position  $p = p'$ , shifts  $c'$  to position  $p + 1$  but does not affect the number of symbols deleted before position  $p'$  (i.e.,  $ND(S, p) = nd'$ ). If  $p > p'$ , the symbol at position  $p - 1$  before executing  $Ins(p', c', nd')$  will be shifted to position  $p$  (i.e.,  $ND(S, p) = ND(S', p - 1)$ ). Similarly, for  $S = S' \bullet [Del(p')]$ , the execution of  $Del(p')$  will delete the symbol at position  $p'$ . If  $p < p'$ , the execution of  $Del(p')$  does not affect the part before position  $p'$  (i.e.,  $ND(S, p) = ND(S', p)$ ). Otherwise, symbols deleted before  $p$  by  $S$  are symbols deleted before  $p + 1$  by  $S'$  plus the one deleted by  $Del(p')$  (i.e.,  $ND(S, p) = ND(S', p + 1) + 1$ ).

As an example, let us compute the number of symbols deleted before position 2, by the sequence  $[Del(2); Ins(2, a); Ins(1, b)]$  (i.e.,  $ND([Del(2); Ins(2, a); Ins(1, b)], 2)$ ). This number is equal to the number of deleted symbols by  $[Del(2); Ins(2, a)]$  before position 1, because  $Ins(1, b)$  will shift the symbol at position 1 to position 2. Therefore,  $ND([Del(2); Ins(2, a); Ins(1, b)], 2) = ND([Del(2); Ins(2, a)], 1)$ . The number of deleted symbols by  $[Del(2); Ins(2, a)]$  before position 1 is equal to  $ND([Del(2)], 1)$  as  $Ins(2, a)$  does not affect symbols before position 2. Finally, since  $1 < 2$ , it follows that  $ND([Del(2)], 1) = ND([], 1) = 0$ . Therefore, there is no symbol deleted by  $[Del(2); Ins(2, a); Ins(1, b)]$  before position 2. Consider now the sequence  $[Ins(2, a); Del(3); Ins(1, b)]$ , which is equivalent to the previous one. Let us compute  $ND([Ins(2, a); Del(3); Ins(1, b)], 2)$ ,

using Definition 1:  $ND([Ins(2, a); Del(3); Ins(1, b)], 2) = ND([Ins(2, a); Del(3)], 1) = ND([Ins(2, a)], 1) = ND([], 1) = 0$ . Note that, the number of deleted symbols before position 2 is the same for the two equivalent sequences.

Since in OT, operations are, in general, executed after integration in different orders, to ensure consistency, our computation procedure of  $ND$  should give the same result for all equivalent sequences and any position  $p$  (i.e.,  $ND(S, p) = ND(S', p)$ , for any pair of equivalent sequences  $S$  and  $S'$ ). We first define our IT function then we establish some nice properties of  $ND$  which will be useful to show consistency of our IT function.

## 4.2 Our IT Function

In our IT function, the signatures of the insert and delete operations are  $Ins(p, c, nd)$  and  $Del(p)$ , respectively, where  $p$  is a position,  $c$  is a symbol and  $nd$  is the number of symbols deleted before position  $p$ . The conflicting situations between two concurrent insert operations are handled using their extra parameters  $nd$ , in a similar way as in other IT functions. More precisely, when transforming a pair of insert operations having the same current position, their parameters  $nd$  are first compared in order to recover the position relation at the generation phase. If their parameters  $nd$  are equal, then their symbols are compared to tie-break an eventual conflict. The IT function proposed here is reported at Fig. 17.

## 4.3 Relationships between Positions and the Extra Parameters

We establish, in Lemma 1 and Theorem 1, some relationships between positions of insert operations generated on the same state and their parameters  $nd$ . We first suppose, in Lemma 1, that the operations are generated after executing the same sequence of operations. Then, we consider, in Theorem 1, the case where the operations are generated after executing different but equivalent sequences (i.e., they consist of the same set of original operations executed, after integration, in different orders). Intuitively, these relationships mean that for any pair of insert operations generated on the same state, the order relation of their positions is the same as the order relation of their extra parameters  $nd$ .

**Lemma 1.** Let  $o_1 = Ins(p_1, c_1, nd_1)$  and  $o_2 = Ins(p_2, c_2, nd_2)$  be two insert operations generated on the same state, just after executing the same sequence of operations  $S$ . Then:  $p_1 < p_2 \Rightarrow nd_1 \leq nd_2$ .

$$\begin{aligned}
IT(Ins(p_1, c_1, nd_1), Ins(p_2, c_2, nd_2)) &= \begin{cases} Ins(p_1, c_1, nd_1) & \text{if } p_1 < p_2 \vee (p_1 = p_2 \wedge nd_1 < nd_2) \\ & \vee (p_1 = p_2 \wedge nd_1 = nd_2 \wedge c_1 \leq c_2) \\ Ins(p_1 + 1, c_1, nd_1) & \text{if } p_1 > p_2 \vee (p_1 = p_2 \wedge nd_1 > nd_2) \\ & \vee (p_1 = p_2 \wedge nd_1 = nd_2 \wedge c_1 > c_2) \end{cases} \\
IT(Ins(p_1, c_1, nd_1), Del(p_2)) &= \begin{cases} Ins(p_1, c_1, nd_1) & \text{if } p_1 \leq p_2 \\ Ins(p_1 - 1, c_1, nd_1 + 1) & \text{otherwise} \end{cases} \\
IT(Del(p_1), Ins(p_2, c_2, nd_2)) &= \begin{cases} Del(p_1) & \text{if } p_1 < p_2 \\ Del(p_1 + 1) & \text{otherwise} \end{cases} \quad IT(Del(p_1), Del(p_2)) = \begin{cases} Del(p_1) & \text{if } p_1 < p_2 \\ Del(p_1 - 1) & \text{if } p_1 > p_2 \\ Nop() & \text{otherwise} \end{cases}
\end{aligned}$$

Fig. 17. The proposed IT function.

**Proof.** By definition,  $nd_1 = ND(S, p_1)$  and  $nd_2 = ND(S, p_2)$ . We show by induction on the length of  $S$  that  $p_1 < p_2 \Rightarrow ND(S, p_1) \leq ND(S, p_2)$ . For  $S = []$ , by definition,  $ND(S, p_1) = ND(S, p_2)$ . For  $S = S' \bullet [o']$ , assume that  $p_1 < p_2 \Rightarrow ND(S', p_1) \leq ND(S', p_2)$  and let us show that  $ND(S, p_1) \leq ND(S, p_2)$ . Let  $p'$  be the position of  $o'$ . We consider 9 cases:

1)  $p_1 < p_2 < p'$ : By definition,  $ND(S, p_1) = ND(S', p_1)$ ,  $ND(S, p_2) = ND(S', p_2)$ . By assumption,  $ND(S', p_1) \leq ND(S', p_2)$ . Then  $ND(S, p_1) \leq ND(S, p_2)$ .

2)  $p_1 < p_2 = p'$  and  $o' = Ins(p', c', nd')$ :  $ND(S, p_1) = ND(S', p_1)$  and  $ND(S, p_2) = nd'$ . Since  $p_1 < p'$ , it follows that  $ND(S', p_1) \leq nd'$  and then  $ND(S, p_1) \leq ND(S, p_2)$ .

3)  $p_1 < p_2 = p'$  and  $o' = Del(p')$ :  $ND(S, p_1) = ND(S', p_1)$  and  $ND(S, p_2) = ND(S', p_2 + 1) + 1$ . Then  $ND(S, p_1) \leq ND(S, p_2)$ .

4)  $p_1 < p' < p_2$  and  $o' = Ins(p', c', nd')$ :  $ND(S, p_1) = ND(S', p_1)$  and  $ND(S, p_2) = ND(S', p_2 - 1)$ . Then  $ND(S, p_1) \leq ND(S, p_2)$ .

5)  $p_1 < p' < p_2$  and  $o' = Del(p')$ :  $ND(S, p_1) = ND(S', p_1)$  and  $ND(S, p_2) = ND(S', p_2 + 1) + 1$ . Then  $ND(S, p_1) \leq ND(S, p_2)$ .

6)  $p_1 = p' < p_2$  and  $o' = Ins(p', c', nd')$ :  $ND(S, p_1) = nd'$  and  $ND(S, p_2) = ND(S', p_2 - 1)$ . Since  $p' \leq p_2 - 1$ , it follows that  $nd' \leq ND(S', p_2 - 1)$  and then  $ND(S, p_1) \leq ND(S, p_2)$ .

7)  $p_1 = p' < p_2$  and  $o' = Del(p')$ :  $ND(S, p_1) = ND(S', p_1 + 1) + 1$  and  $ND(S, p_2) = ND(S', p_2 + 1) + 1$ . It follows that  $ND(S, p_1) \leq ND(S, p_2)$ .

8)  $p' < p_1 < p_2$  and  $o' = Ins(p', c', nd')$ :  $ND(S, p_1) = ND(S', p_1 - 1)$  and  $ND(S, p_2) = ND(S', p_2 - 1)$ . Then  $ND(S, p_1) \leq ND(S, p_2)$ .

9)  $p' < p_1 < p_2$  and  $o' = Del(p')$ :  $ND(S, p_1) = ND(S', p_1 + 1) + 1$  and  $ND(S, p_2) = ND(S', p_2 + 1) + 1$ . Then  $ND(S, p_1) \leq ND(S, p_2)$ .  $\square$

**Theorem 1.** Let  $o_1 = Ins(p_1, c_1, nd_1)$  and  $o_2 = Ins(p_2, c_2, nd_2)$  be two insert operations generated on the same state after

executing two different but equivalent sequences of operations  $S_1$  and  $S_2$ . Then: 1)  $p_1 = p_2 \Rightarrow nd_1 = nd_2$  and 2)  $p_1 < p_2 \Rightarrow nd_1 \leq nd_2$ .

**Proof.** By definition  $nd_1 = ND(S_1, p)$  and  $nd_2 = ND(S_2, p)$ .

1) By assumption, sequences  $S_1$  and  $S_2$  consist of the same set of original operations executed, after integration, in two different orders. The different execution orders can be obtained by successive pairwise permutations of concurrent operations. If one permutation of concurrent operations preserves the value given for the number of deletes, then any sequence of such swaps preserve also this value. As a result, it is sufficient to consider only the case of one permutation:  $S_1 = S \bullet [o_1; o_{21}] \bullet S'$  and  $S_2 = S \bullet [o_2; o_{12}] \bullet S'$ , where  $o_{21} = IT(o_2, o_1)$  and  $o_{12} = IT(o_1, o_2)$ . So, to show that  $ND(S_1, p) = ND(S_2, p)$ , it suffices to show that  $ND(S \bullet [o_1; o_{21}], p) = ND(S \bullet [o_2; o_{12}], p)$ . Table 3 reports values of  $ND(S \bullet [o_1; o_{21}], p)$  and  $ND(S \bullet [o_2; o_{12}], p)$  for every transformation case (see Table 2). They are equal for each transformation case.

2) According to Lemma 1, if  $p_1 < p_2$  then  $ND(S_1, p_1) \leq ND(S_1, p_2)$ . Part 1) of Theorem 1 states that  $ND(S_1, p_1) = ND(S_2, p_1)$  and  $ND(S_1, p_2) = ND(S_2, p_2)$ . It follows that:  $p_1 < p_2 \Rightarrow ND(S_1, p_1) \leq ND(S_2, p_2)$ .  $\square$

Concretely, these nice relationships established above ensure some consistency to the computation procedure of  $ND$  given in Definition 1. Indeed, all insert operations with the same position, generated on the same state at different sites, will have the same parameter  $nd$ , even if this state has been reached by different but equivalent sequences. Furthermore, if the inserting position  $p_1$  of an operation  $o_1$  is on the left of the inserting position  $p_2$  of another operation  $o_2$ , generated on the same state, then the number of symbols deleted before  $p_1$  is less or equal to the number of symbols deleted before  $p_2$  (i.e.,  $nd_1 \leq nd_2$ ). Note that  $nd_1 = nd_2$  in case there is no deleted symbol between  $p_1$  and  $p_2$ , including

TABLE 2  
Transformation Cases for  $IT(o_1, o_2)$  and  $IT(o_2, o_1)$

| k | IT cases for $IT(o_1, o_2)$ and $IT(o_2, o_1)$                                                                              |
|---|-----------------------------------------------------------------------------------------------------------------------------|
| 0 | $p_1 < p_2$                                                                                                                 |
| 1 | $p_1 > p_2$                                                                                                                 |
| 2 | $o_1 = Ins(p_1, c_1, nd_1) \wedge o_2 = Ins(p_2, c_2, nd_2) \wedge p_1 = p_2 \wedge nd_1 = nd_2 \wedge c_1 < c_2$           |
| 3 | $o_1 = Ins(p_1, c_1, nd_1) \wedge o_2 = Ins(p_2, c_2, nd_2) \wedge p_1 = p_2 \wedge nd_1 = nd_2 \wedge c_1 > c_2$           |
| 4 | $o_1 = Ins(p_1, c_1, nd_1) \wedge o_2 = Ins(p_2, c_2, nd_2) \wedge p_1 = p_2 \wedge nd_1 < nd_2$                            |
| 5 | $o_1 = Ins(p_1, c_1, nd_1) \wedge o_2 = Ins(p_2, c_2, nd_2) \wedge p_1 = p_2 \wedge nd_1 > nd_2$                            |
| 6 | $o_1 = Ins(p_1, c_1, nd_1, s_1) \wedge o_2 = Ins(p_2, c_2, nd_2, s_2) \wedge p_1 = p_2 \wedge nd_1 = nd_2 \wedge c_1 = c_2$ |
| 7 | $(o_1 = Del(p_1) \vee o_2 = Del(p_2)) \wedge p_1 = p_2$                                                                     |

TABLE 3  
Computing  $ND(S \bullet [o_1; o_{21}], p)$  and  $ND(S \bullet [o_2; o_{12}], p)$

| Transformation cases                                             | $ND(S \bullet [o_1; o_{21}], p)$                                                                                                                                                                            | $ND(S \bullet [o_2; o_{12}], p)$                                                                                                                                                                            |
|------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $Ins, Ins, k \in \{0, 2, 4, 6\}, nd_{12} = nd_1, nd_{21} = nd_2$ | $\begin{cases} ND(S, p) & \text{if } p < p_1 \\ nd_1 & \text{if } p = p_1 \\ ND(S, p-1) & \text{if } p_1 < p \leq p_2 \\ nd_{21} & \text{if } p = p_{21} \\ ND(S, p-2) & \text{if } p > p_{21} \end{cases}$ | $\begin{cases} ND(S, p) & \text{if } p < p_1 \\ nd_{12} & \text{if } p = p_1 \\ ND(S, p-1) & \text{if } p_1 < p \leq p_2 \\ nd_2 & \text{if } p = p_{21} \\ ND(S, p-2) & \text{if } p > p_{21} \end{cases}$ |
| $Ins, Ins, k \in \{1, 3, 5\}, nd_{12} = nd_1, nd_{21} = nd_2$    | $\begin{cases} ND(S, p) & \text{if } p < p_2 \\ nd_{21} & \text{if } p = p_2 \\ ND(S, p-1) & \text{if } p_2 < p \leq p_1 \\ nd_1 & \text{if } p = p_{12} \\ ND(S, p-2) & \text{if } p > p_{12} \end{cases}$ | $\begin{cases} ND(S, p) & \text{if } p < p_2 \\ nd_2 & \text{if } p = p_2 \\ ND(S, p-1) & \text{if } p_2 < p \leq p_1 \\ nd_{12} & \text{if } p = p_{12} \\ ND(S, p-2) & \text{if } p > p_{12} \end{cases}$ |
| $Del, Del, k = 0$                                                | $\begin{cases} ND(S, p) & \text{if } p < p_1 \\ ND(S, p+1) + 1 & \text{if } p_1 \leq p < p_{21} \\ ND(S, p+2) + 2 & \text{if } p \geq p_{21} \end{cases}$                                                   | $\begin{cases} ND(S, p) & \text{if } p < p_1 \\ ND(S, p+1) + 1 & \text{if } p_1 \leq p < p_{21} \\ ND(S, p+2) + 2 & \text{if } p \geq p_{21} \end{cases}$                                                   |
| $Del, Del, k = 1$                                                | $\begin{cases} ND(S, p) & \text{if } p < p_2 \\ ND(S, p+1) + 1 & \text{if } p_2 \leq p < p_{12} \\ ND(S, p+2) + 2 & \text{if } p \geq p_{12} \end{cases}$                                                   | $\begin{cases} ND(S, p) & \text{if } p < p_2 \\ ND(S, p+1) + 1 & \text{if } p_2 \leq p < p_{12} \\ ND(S, p+2) + 2 & \text{if } p \geq p_{12} \end{cases}$                                                   |
| $Del, Del, k = 7$                                                | $ND(S \bullet [o_1], p)$                                                                                                                                                                                    | $ND(S \bullet [o_2], p)$                                                                                                                                                                                    |
| $Ins, Del, k \in \{0, 7\}$                                       | $\begin{cases} ND(S, p) & \text{if } p < p_1 \\ nd_1 & \text{if } p = p_1 \\ ND(S, p-1) & \text{if } p_1 < p \leq p_2 \\ ND(S, p) + 1 & \text{if } p \geq p_{12} \end{cases}$                               | $\begin{cases} ND(S, p) & \text{if } p < p_1 \\ nd_{12} & \text{if } p = p_1 \\ ND(S, p-1) & \text{if } p_1 < p \leq p_2 \\ ND(S, p) + 1 & \text{if } p \geq p_{12} \end{cases}$                            |
| $Ins, Del, k = 1$                                                | $\begin{cases} ND(S, p) & \text{if } p < p_2 \\ ND(S, p+1) + 1 & \text{if } p_2 \leq p < p_1 \\ nd_1 + 1 & \text{if } p = p_{12} \\ ND(S, p) + 1 & \text{if } p \geq p_1 \end{cases}$                       | $\begin{cases} ND(S, p) & \text{if } p < p_2 \\ ND(S, p+1) + 1 & \text{if } p_2 \leq p < p_1 \\ nd_{12} & \text{if } p = p_{12} \\ ND(S, p) + 1 & \text{if } p \geq p_1 \end{cases}$                        |

position  $p_1$ . This extra parameter seems then to be appropriate to handle conflicting operations.

Note that the above theorem is very important in the sense that it enables us to avoid recomputing the parameter  $nd$  of an insert operation in the remote sites. It suffices to compute this parameter when the insert operation is generated and then include it in the operation sent to all other sites.

#### 4.4 Proof of Consistency

To prove consistency, it suffices to show that our IT satisfies both properties TP1 and TP2. In [15], the authors proposed a formal framework for modelling and verifying IT functions with algebraic specifications. For checking the properties TP1 and TP2, they used an automatic theorem proving. However, this theorem proving approach has some shortcomings: (i) the model of the system is sound but not complete w.r.t. TP1 and TP2 (i.e., it does not guarantee that the violation of property TP1 or TP2 is really feasible); (ii) there is no guidance to understand the counterexamples (when the properties are not verified); (iii) it requires some interaction (by injecting new lemmas) to complete the verification. In [18], the authors addressed these drawbacks and proposed a symbolic model-checking technique based on difference bound matrices (DBMs) to verify whether an IT function satisfies properties TP1 and TP2. The verification of these properties is performed automatically and symbolically without carrying out different copies of the shared object and executing explicitly

the updates. Moreover, unlike the approach proposed in [15], the approach proposed in [18] provides faisable and complete symbolic counterexamples. As in [18], we use a symbolic model-checking technique, where the shared objects are abstracted and their update operations are handled symbolically using difference bound matrices. In our context, DBMs are used to encode sets of constraints of the form  $p_i - p_j \leq c$ , where  $p_i, p_j$  are integer variables and  $c$  is an integer constant. From the practical point of view, a DBM is a square matrix  $P$  indexed by variables. Each entry  $P_{ij}$  represents the atomic constraint  $p_i - p_j \leq P_{ij}$ . If there is no upper bound on  $p_i - p_j$  with  $i \neq j$ ,  $P_{ij}$  is set to  $\infty$ . Entry  $P_{ii}$  is set to 0. Constraints  $p_i - p_j = c$  and  $p_i - p_j \geq c$  are considered as abbreviations of atomic constraints. In the following, we use invariantly atomic constraints or their abbreviations.

A set of constraints is consistent iff they represent a non empty domain. Although the same non empty domain may be encoded by different DBMs, they have a canonical form. The canonical form of a DBM is the representation with tightest bounds on all differences between variables, computed by propagating the effect of each entry through the DBM. Canonical forms are much more useful to verify consistency and test of equivalence.

Two sets of atomic constraints are equivalent iff the canonical forms of their DBMs are identical. To verify the consistency of a DBM (i.e., a set of atomic constraints), it suffices to apply a shortest-path algorithm and to stop the algorithm as soon as a negative cycle is detected. The presence

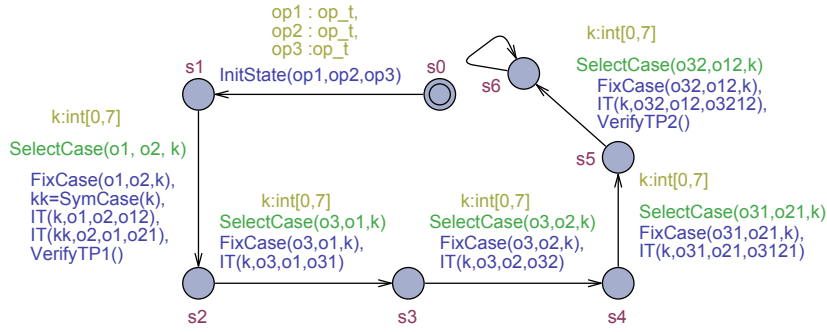


Fig. 18. Automaton used to verify TP1 and TP2.

of negative cycles means that the set of atomic constraints is inconsistent.

The model depicted at Fig. 18 is used to verify whether or not our IT function satisfies properties TP1 and TP2. The model starts by selecting types of the three update operations  $o_1$ ,  $o_2$  and  $o_3$  and initializing the DBMs of their parameters (initial DBMs are free from constraints). This is the role of the edge  $(s_0, s_1)$ . Afterwards, the model successively selects for each operational transformation, a feasible transformation case among the eight cases shown at Table 2, fixes and applies the transformation case, and verifies TP1 or TP2 as soon as the needed transformations are computed. For instance, the edge  $(s_1, s_2)$  selects a transformation case  $k$  for  $o_1$  with regard to  $o_2$  ( $SelectCase(o_1, o_2, k)$ ), fixes the selected case ( $FixCase(o_1, o_2, k)$ ) and applies it to transform  $o_1$  with regard to  $o_2$ . The symmetrical case  $kk = SymCase(k)$  is used to transform  $o_2$  with regard to  $o_1$ . After these transformations ( $IT(k, o_1, o_2, o_{12})$  and  $IT(kk, o_2, o_1, o_{21})$ ), the property TP1 is verified. Concretely, function  $FixCase(o_i, o_j, k)$  adds to the DBMs the constraints corresponding to the transformation case  $k$  to be applied to  $o_i$  with regard to  $o_j$ . Function  $IT(k, o_i, o_j, o_{ij})$  applies the transformation case  $k$  to transform  $o_i$  with regard to  $o_j$  and returns the transformation result in  $o_{ij}$ . Functions  $verifyTP1()$  and  $verifyTP2()$  set in boolean variables  $TP1$  and  $TP2$  the results of verification of properties TP1 and TP2, respectively. Function  $verifyTP1()$  verifies whether or not the domain of  $P$  satisfies one of the conditions of equivalence given in Table 5<sup>4</sup> accordingly to the types of operations  $o_{12}$  and  $o_{21}$ . For instance, if  $o_{12}$  and  $o_{21}$  are insert operations, TP1 is satisfied iff  $P = P \cup \{p_{12} = p_1 \leq p_2 = p_{21} - 1\}$  or  $P = P \cup \{p_{21} = p_2 \leq p_1 = p_{12} - 1\}$  or  $([o_1] \equiv [o_2] \wedge [o_{12}] \equiv [o_{21}])$ . Function  $verifyTP2()$  verifies whether or not operations  $o_{3121}$  and  $o_{3212}$  are identical, i.e., they are of the same type and have the same parameters. In case  $o_{3121}$  and  $o_{3212}$  are not of type *Not*, then they have identical position parameter iff  $P = P \cup \{p_{3121} = p_{3212}\}$ , which means that  $p_{3121} = p_{3212}$  holds in the whole domain of  $P$ .

For example, suppose that three insert operations are selected by the edge  $(s_0, s_1)$ . In this case, three DBMs  $P$ ,  $Nd$  and  $C$  over  $\{p_1, p_2, p_3\}$ ,  $\{nd_1, nd_2, nd_3\}$  and  $\{c_1, c_2, c_3\}$  are created. The initial domain of each DBM is  $\mathbb{N}^3$ ,  $\mathbb{N}$  being the set of non negative integers. Suppose now that for the

operational transformation of  $o_1$  with regard to  $o_2$ , the selected  $k$  is 2. The operational transformation case of  $o_2$  with regard to  $o_1$  is then  $kk = SymCase(2) = 3$ . In this case,  $FixCase(o_1, o_2, k)$  adds sets of constraints  $\{p_1 = p_2\}$ ,  $\{nd_1 = nd_2\}$  and  $\{c_1 < c_2\}$  to  $P$ ,  $Nd$  and  $C$ , respectively (see Table 2). The operational transformations of  $o_1$  with regard to  $o_2$  and  $o_2$  with regard to  $o_1$  (i.e.,  $IT(k, o_1, o_2, o_{12})$ ,  $(kk, o_2, o_1, o_{21})$ ) create two insert operations  $o_{12}$  and  $o_{21}$ , add to  $P$ ,  $Nd$  and  $C$  sets of constraints  $\{p_{12} = p_1, p_{21} = p_2 + 1\}$ ,  $\{nd_{12} = nd_1, nd_{21} = nd_2\}$  and  $\{c_{12} = c_1, c_{21} = c_2\}$ , respectively. Finally,  $verifyTP1()$  states that the property TP1 is satisfied. Afterwards, the same procedure is repeated for computing the operational transformations needed to verify TP2 (i.e.,  $o_{3121} = o_{3212}$ ).

We have used the tool UPPAAL [24] to verify whether or not our model satisfies the safety properties  $AG TP1$  and  $AG TP2$ . UPPAAL states that the first property is satisfied (i.e., the proposed IT satisfies TP1). It concludes however that the second property is not satisfied. The only counterexamples provided are scenarios where there are at least two insert operations  $o_1 = Ins(p_1, c_1, nd_1)$  and  $o_2 = Ins(p_2, c_2, nd_2)$  generated at the same state s.t.  $p_1 \leq p_2 \wedge nd_1 > nd_2$  or  $p_1 \geq p_2 \wedge nd_1 < nd_2$ . According to Lemma 1 and Theorem 1 such scenarios are infeasible. When we exclude such infeasible scenarios, UPPAAL concludes that property TP2 is satisfied. Therefore, the proposed IT satisfies TP1 and TP2.

## 4.5 Comparison

In this section, we give comparison between our IT function and the function given in [22]. It is well known that the main issue for satisfying TP2 is due to the semantics of the delete operation. Recall that this operation removes an element at a given position and decreases the length of the shared document. That is why, in [22], the authors tackled the TP2 problem by changing the semantics of the deletion operation. Indeed, operation  $Del(p)$  does not remove the element at position  $p$  but makes it invisible. This new semantics leads the authors to manage two distinct states, called *view* and *data* models. The view model is seen by the user and contains only visible characters while the data model contains all characters: characters displayed in the view model and the hidden characters (called *tombstones*) resulting from delete operations. Any generated update operation involves two updates from view and data models, respectively. As illustrated in Fig. 19, consider the operation  $Ins(3, y)$  related to the view model. The corresponding

4. We suppose that if  $o_{12}$  (resp.  $o_{21}$ ) is an insert operation then  $c_{12} = c_1$  (resp.  $c_{21} = c_2$ ).

TABLE 4  
Condition of Equivalence of  $[o_1; o_{21}]$  and  $[o_2; o_{12}]$

| Type of $o_{12}$ | Type of $o_{21}$ | $[o_1; o_{21}] \equiv [o_2; o_{12}]$                 | Type of $o_{12}$ | Type of $o_{21}$ | $[o_1; o_{21}] \equiv [o_2; o_{12}]$ |
|------------------|------------------|------------------------------------------------------|------------------|------------------|--------------------------------------|
| <i>Ins</i>       | <i>Ins / Del</i> | $p_{12} = p_1 \leq p_2 = p_{21} - 1$                 | <i>Ins / Del</i> | <i>Ins</i>       | $p_{21} = p_2 \leq p_1 = p_{12} - 1$ |
| <i>Del</i>       | <i>Ins / Del</i> | $p_{12} = p_1 < p_2 = p_{21} + 1$                    | <i>Ins / Del</i> | <i>Del</i>       | $p_{21} = p_2 < p_1 = p_{12} + 1$    |
| -                | -                | $[o_1] \equiv [o_2] \wedge [o_{21}] \equiv [o_{12}]$ |                  |                  |                                      |

TABLE 5  
Complexity Comparison

| Operation     | TTF's solution [22]                                                                                 | Our solution                                        |
|---------------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| <i>Insert</i> | $O(m)$ where $m$ is the length of the data model.<br>The size of the data model will never decrease | $O(n)$ where $n$ is the length of the local history |
| <i>Delete</i> | $O(m)$ where $m$ is the length of the data model                                                    | $O(1)$                                              |

position in the data model is processed by finding out the location of the 3th visible character, after skipping each possible hidden character encountered. The search process relies on three visible characters. Hence, the corresponding operation related to the data model is  $Ins(5, y)$ . The hidden characters  $h$  and  $n$  located at position 1 and 4, respectively in the data model, are skipped.

Although the authors presented a correct set of transformation functions, the length of the sequence is always increased (but never decreased), as the new semantics of deletion operation has no effect on the document length. In addition, their solution needs some extra procedures to manage two distinct states due to the hidden characters, namely: the view (the state seen by the user) and the model (the persistent state). Unlike this situation, our proposed transformation function considers the natural semantics of deletion operation as suggested by Ellis and Gibbs in [2] and our deletion operation removes physically the character and reduces the document length. Our transformation function is based on the parameter  $nd$  filled with the number of deleted symbols to maintain consistency without any particular data model besides. As for complexities at generation time, a comparison is given in Table 5. Even though, the overall complexity is linear for both approaches. Unlike TTF [22], it should be noted that for our approach, the complexity of delete operation is always constant. Note also that the parameter  $nd$  is computed at the generation step thanks to Theorem 1, there is no need to recompute it when it is received by a remote site.

In Wu and Pui [25], the authors have combined CRDT [3], [4] with TTF [22]. Their approach is based on a data structure called Partial Persistent Sequences (PPS), where each item of the shared object has a unique position identifier, i.e., a rational number, called position stamp. It is then always possible to allocate a new position stamp to any item to be inserted into the object. As in TTF [22], when an item is removed from the object, it is hidden and its position stamp is mapped to an empty item. To

prevent situations where different users compute the same position concurrently, Wu et al. have proposed in [26] to allocate in advance, to each user, sub-ranges of position identifiers. The range between two position identifiers is partitioned into sub-ranges, dividing the distance by the number of users. Between two given position stamps, a unique sub-range is assigned to each user. Even if the management of the position identifiers is improved, it remained non-optimal due to waste of the identifier space such as some ranges could be reserved but never used. In addition, the identifier space will increase rapidly. To deal with the rebalancing issue, Wu et al. have proposed in [26] to remove the position identifiers of deleted items followed by a computation of new position stamps values for the non-removed items. As any CRDT, this approach does not need to satisfy the property TP2, however, its correctness depends on the uniqueness of position identifiers. The authors do not show clearly how to achieve the uniqueness of position identifiers.

## 5 CONCLUSION

In this paper, we have investigated, using the controller synthesis technique, whether or not there exist IT functions which ensure data convergence, based on the operational transformation framework. In this framework, an IT function ensures convergence iff it satisfies two properties TP1 and TP2. We have shown that there are some IT functions for the basic signatures of insert and delete operations, which satisfy TP1 but there is no IT function, which satisfies TP2. We have then identified two pairwise scenarios, which prevent to get a consistent function IT (with no one-to-many relation). These scenarios were useful to find an appropriate extra parameter for the insert operation and to define, based on this extra parameter, a new IT. The role of the extra parameter is to record the number of deleted symbols before the inserting position. This parameter is computed when the operation is generated and then updated whenever it is transformed against a delete operation with smaller position.

The consistency of our IT function is formally proved by means of a symbolic model checking technique, where all parameters of the update operations are handled symbolically using difference bound matrices. Initially, the verification of properties TP1 and TP2 is performed without imposing any relationship between parameters of the update operations (i.e., an over-approximation of the

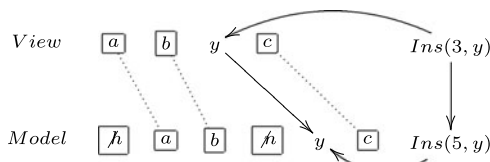


Fig. 19. View and Model states (Figure taken from [22]).

effective model). It concludes that TP1 is satisfied but TP2 is not satisfied. The counterexamples provided for property TP2 are scenarios where there are at least two insert operations  $o_1 = \text{Ins}(p_1, c_1, nd_1)$  and  $o_2 = \text{Ins}(p_2, c_2, nd_2)$  generated at the same state s.t.  $p_1 \leq p_2 \wedge nd_1 > nd_2$  or  $p_1 \geq p_2 \wedge nd_1 < nd_2$ . According to Lemma 1 and Theorem 1 such scenarios are infeasible. When we exclude such infeasible scenarios, the verification process concludes that property TP2 is satisfied. Therefore, the proposed IT satisfies both TP1 and TP2.

In the near future, we will apply our IT to a specific system as peer-to-peer collaborative editors and study performance issue. We also plan to investigate the existence of consistent IT functions, in case of shared object with non linear structure, using the controller synthesis and model checking techniques.

## REFERENCES

- [1] P.A. Bernstein and N. Goodman, "Multiversion Concurrency Control: Theory and Algorithms," *ACM Trans. Database Systems*, vol. 8, pp. 465-483, 1983.
- [2] C.A. Ellis and S.J. Gibbs, "Concurrency Control in Groupware Systems," *Proc. ACM SIGMOD Conf. Management of Data*, pp. 399-407, 1989.
- [3] N. Preguiça, J. Marques, M. Shapiro, and M. Letia, "A Commutative Replicated Data Type for Cooperative Editing," *Proc. 29th IEEE Int'l Conf. Distributed Computing Systems*, pp. 395-403, 2009.
- [4] S. Weiss, P. Urso, and P. Molli, "Logoot-Undo: Distributed Collaborative Editing System on P2P Networks," *IEEE Trans. Parallel and Distributed Systems*, vol. 21, no. 8, pp. 1162-1174, Aug. 2010.
- [5] S. Weiss, P. Urso, and P. Molli, "Logoot: A Scalable Optimistic Replication Algorithm for Collaborative Editing on P2P Networks," *29th IEEE Int'l Conf. Distributed Computing Systems*, pp. 404-412, 2009.
- [6] G. Oster, P. Urso, P. Molli, and A. Imine, "Data Consistency for P2P Collaborative Editing," *Proc. 20th Conf. Computer Supported Cooperative Work*, pp. 259-268, 2006.
- [7] M. Ressel, D. Nitsche Ruhland, and R. Gunzenhauser, "An Integrating, Transformation-Oriented Approach to Concurrency Control and Undo in Group Editors," *Proc. ACM Computer Supported Cooperative Work (CSCW '96)*, pp. 288-297, 1996.
- [8] M. Suleiman, M. Cart, and J. Ferrié, "Concurrent Operations in a Distributed and Mobile Collaborative Environment," *Proc. IEEE Data Eng. (ICDE '98)*, pp. 36-45, 1998.
- [9] N. Vidot, M. Cart, J. Ferrié, and M. Suleiman, "Copies Convergence in a Distributed Real-Time Collaborative Environment," *Proc. ACM Computer Supported Cooperative Work (CSCW '00)*, 2000.
- [10] C. Sun and C. Ellis, "Operational Transformation in Real-Time Group Editors: Issues, Algorithms, and Achievements," *Proc. ACM Computer Supported Cooperative Work (CSCW '98)*, pp. 59-68, 1998.
- [11] D. Sun and C. Sun, "Context-Based Operational Transformation for Distributed Collaborative Editing Systems," *IEEE Trans. Parallel and Distributed Systems*, vol. 20, no. 10, pp. 1454-1470, Oct. 2009.
- [12] C. Sun, X. Jia, Y. Zhang, Y. Yang, and D. Chen, "Achieving Convergence, Causality-Preservation and Intention-Preservation in Real-Time Cooperative Editing Systems," *ACM Trans. Computer-Human Interaction*, vol. 5, no. 1, pp. 63-108, 1998.
- [13] M. Suleiman, M. Cart, and J. Ferrié, "Serialization of Concurrent Operations in a Distributed Collaborative Environment," *Proc. Int'l ACM Conf. Supporting Group Work: The Integration Challenge (GROUP '97)*, pp. 435-445, 1997.
- [14] A. Imine, P. Molli, G. Oster, and M. Rusinowitch, "Proving Correctness of Transformation Functions in Real-Time Groupware," *Proc. Eighth European Conf. Computer Supported Cooperative Work (ECSCW '03)*, 2003.
- [15] A. Imine, M. Rusinowitch, G. Oster, and P. Molli, "Formal Design and Verification of Operational Transformation Algorithms for Copies Convergence," *Theoretical Computer Science*, vol. 351, no. 2, pp. 167-183, 2006.
- [16] A. Imine, "Conception Formelle d'Algorithmes de Réplication Optimiste vers l'Édition Collaborative dans les Réseaux Pair-à-Pair," PhD Thesis, Univ. of Henri Poincaré, 2006.
- [17] H. Boucheneb and A. Imine, "On Model-Checking Optimistic Replication Algorithms," *Proc. Int'l Conf. FMOODS/FORTE*, pp. 73-89, 2009.
- [18] H. Boucheneb, A. Imine, and M. Najem, "Symbolic Model-Checking of Optimistic Replication Algorithms," *Proc. Int'l Conf. Integrated Formal Methods (IFM '10)*, pp. 89-104, 2010.
- [19] C. Sun, S. Xia, D. Sun, D. Chen, H. Shen, and W. Cai, "Transparent Adaptation of Single-User Applications for Multi-User Real-Time Collaboration," *ACM Trans. Computer-Human Interaction*, vol. 13, no. 4, pp. 531-582, 2006.
- [20] A. Imine, "Coordination Model for Real-Time Collaborative Editors," *Proc. 11th Int'l Conf. COORDINATION*, pp. 225-246, 2009.
- [21] D. Li and R. Li, "An Admissibility-Based Operational Transformation Framework for Collaborative Editing Systems," *Computer Supported Cooperative Work*, vol. 19, no. 1, pp. 1-43, 2010.
- [22] G. Oster, P. Molli, P. Urso, and A. Imine, "Tombstone Transformation Functions for Ensuring Consistency in Collaborative Editing Systems," *Proc. Int'l Conf. Collaborative Computing: Networking, Applications and Worksharing (CollaborateCom)*, pp. 1-10, 2006.
- [23] F. Cassez, A. David, E. Fleury, K.G. Larsen, and D. Limei, "Efficient On-the-Fly Algorithms for the Analysis of Timed Games," *Proc. 16th Int'l Conf. CONCUR—Concurrency Theory (CONCUR '05)*, pp. 60-80, 2005.
- [24] K. Larsen, P. Pettersson, and W. Yi, "Uppaal in a Nutshell," *J. Software Tools for Technology Transfer*, vol. 1, no. 1-2, pp. 134-152, 1997.
- [25] Q. Wu and C. Pui, "Consistency in Real-Time Collaborative Editing Systems Based on Partial Persistent Sequences," *technical report, Georgia Inst. of Technology*, 2009.
- [26] Q. Wu, C. Pui, and J.E. Ferreira, "A Partial Persistent Data Structure to Support Consistency in Real-Time Collaborative Editing," *Proc. Int'l Conf. Data Eng. (ICDE)*, pp. 776-779, 2010.



**Aurel Randolph** received the bachelor's engineering degree from the Université Nationale du Bénin in 2000, and the master's degrees in business administration and in computer engineering and applied sciences from the Université d'Abomey-Calavi, Bénin, in 2005 and 2009, respectively. He is currently working toward the PhD degree in computer engineering at École Polytechnique de Montréal, Canada. His research interests include formal methods, model-checking, controller synthesis, software verification, design and verification of real-time systems, design and verification of security policies, design and verification of secure mobile computing systems.



**Hanifa Boucheneb** is currently a full professor at the Department of Computer Engineering of École Polytechnique de Montréal. Her research interests include formal verification techniques of real time systems, time abstractions and partial order techniques. She coauthored more than hundred research papers and served as a PC member/external reviewer in several international journals, conferences and workshops.



**Abdessamad Imine** received the MSc and PhD degrees in computer science from the University of Sciences and Technology of Oran, Algeria, and the University Henri Poincaré of Nancy, France, respectively. He is currently an associate professor at Lorraine University and a senior researcher at INRIA-LORIA center of Nancy. His research interests include optimistic protocols, security for collaborative systems, social networks, and formal methods.



**Alejandro Quintero** received the engineer's degree in computer engineering from Los Andes, Colombia, in 1983. In June 1989, and in 1993, he received the diploma of advanced studies, and the PhD degree in computer engineering, respectively, from the INPG Grenoble and Joseph Fourier University, Grenoble, France. He is currently a full professor at the Department of Computer Engineering of École Polytechnique de Montréal, Canada. His main research interests include services and applications related to mobile computing, sensor networks, network security, and next generation mobile networks. He is the co-author of one book, as well as more than 60 other technical publications including journal and proceedings papers.

▷ **For more information on this or any other computing topic, please visit our Digital Library at [www.computer.org/publications/dlib](http://www.computer.org/publications/dlib).**